

DOCUMENTAȚIE TEHNICĂ

Proiect de Fizică - Simulator interactiv de forțe

Tema: Forța de frecare și rezistența mediului

Autor: Cristi • Referat scris + simulator fizic 2D interactiv

Proiectul live (online)

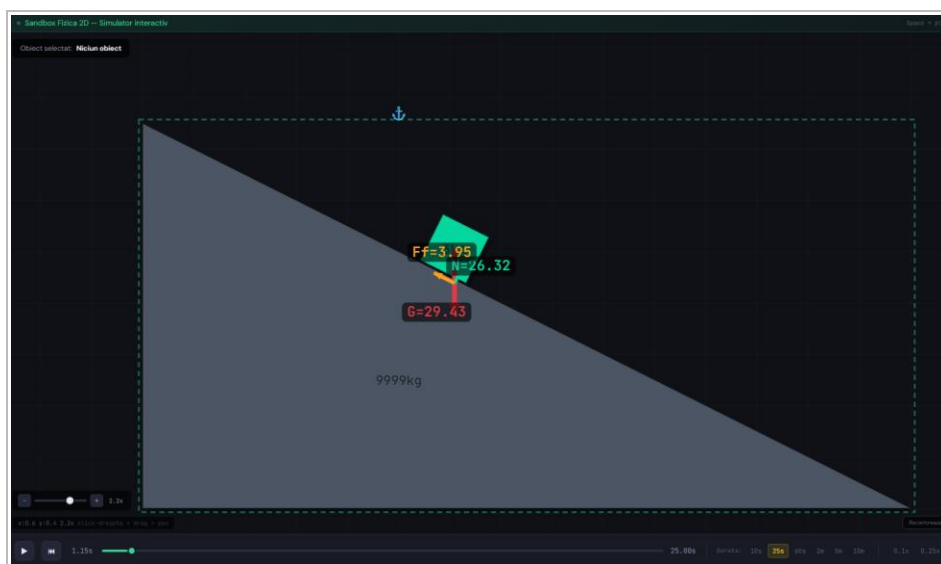
Referat: <https://notcristi.github.io/ProiectFizica/referat.html>

Simulator: <https://notcristi.github.io/ProiectFizica/index.html>

1. Despre ce este proiectul

Proiectul are **două părți care lucrează împreună**. Prima este **referatul** - lucrarea scrisă de fizică despre forța de frecare și rezistența aerului, care conține teorie, formule, experimente și concluzii. A doua este **simulatorul** - un mic laborator de fizică în care pot fi așezate obiecte (cuburi, sfere, rampe), pornită simularea și văzute în timp real toate forțele, vitezele și accelerațiile.

Partea specială este că cele două sunt **conectate**: referatul încorporează simulatorul direct în pagină, așa că atunci când citești despre „planul înclinat” poți vedea chiar acolo experimentul rulând automat. Practic, teoria și experimentul stau una lângă cealaltă pe aceeași pagină.



Captura 1 - Simulatorul: un corp pe planul înclinat, cu forțele G , N și F_f măsurate și afișate în timp real.

2. Ce tehnologii s-au folosit

Aplicația rulează în browser și este construită cu unelte moderne de programare web. Mai jos sunt principalele „cărămizi” din care e făcut proiectul, cu rolul fiecăreia:

 TypeScript	 Node.js	 Vite	 HTML5
 Canvas 2D	 Planck.js	 Export Excel	 Web / Browser

Tehnologie	Versiune	La ce folosește
TypeScript	5.4	Limbajul în care e scris codul (JavaScript cu tipuri, mod strict).
Vite	5.2	Unealta care pornește serverul de lucru și împachetează proiectul.
vite-plugin-singlefile	2.0	Pune tot (JS + CSS) într-un singur fișier HTML la final.
Planck.js	1.4.3	Motorul fizic - rezolvă coliziunile (port JS al lui Box2D).
SheetJS (xlsx)	0.18.5	Exportă datele simulării în fișiere Excel (.xlsx).
Canvas 2D	-	Desenarea scenei pe ecran (API standard din browser).
Google Fonts	-	Fonturile folosite în interfață (Nunito, JetBrains Mono).

De ce aceste alegeri

- **Planck.js** face partea grea - calculează coliziunile dintre obiecte. Peste el, proiectul adaugă fizica corectă de școală (forțe, frecare, grafice).
- **Vite** împachetează totul, la final, într-un **singur fișier HTML** (~690 KB) care se deschide în orice browser, fără server și fără instalare.
- **TypeScript** prinde greșelile încă dinainte de a rula programul, ceea ce face codul mai sigur.

Nu s-a folosit niciun framework greu (React, Vue). Interfața e făcută direct, ceea ce ține proiectul ușor și rapid.

3. Cum funcționează fizica din simulator

Aceasta este „inima” proiectului. Motorul calculează ciocnirile, iar peste el este adăugată **fizica reală din programa școlară**.

3.1. Cele șase forțe simulate

Simulatorul calculează și afișează șase forțe - exact cele studiate la fizică:

- **Greutatea** $G = m \cdot g$
- **Forța normală** N (apăsarea suprafeței)
- **Forța de frecare** $F_f = \mu \cdot N$
- **Rezistența aerului** $F_d = \frac{1}{2} \cdot \rho \cdot C_d \cdot A \cdot v^2$ - direct legată de tema referatului
- **Forța aplicată** F_{apl} , sub un unghi α
- **Tensiunea / forța elastică** din sfori, resorturi și scripeți

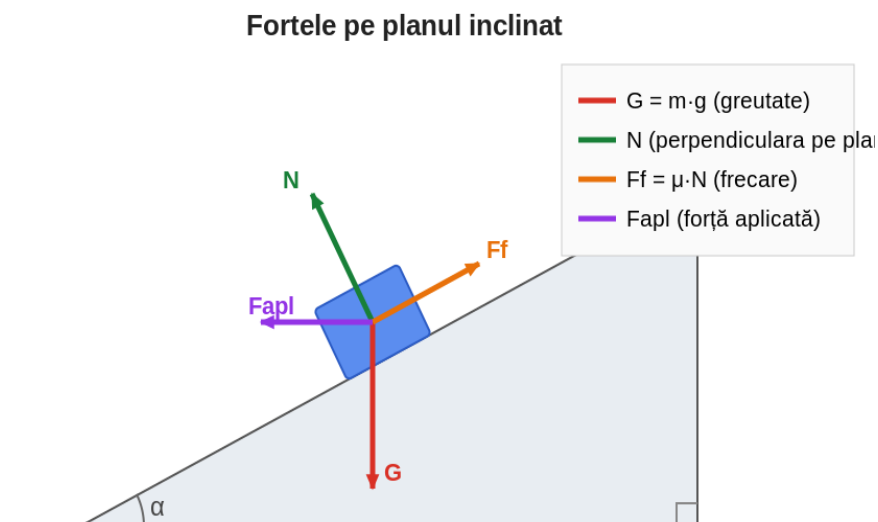
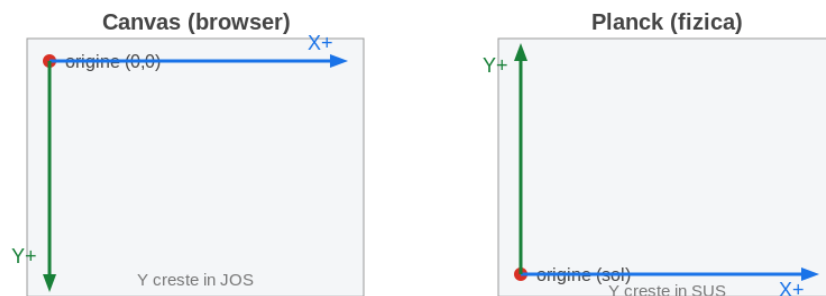


Figura 1 - Forțele care acționează asupra unui corp pe planul înclinat.

3.2. Două sisteme de coordonate

Browserul și motorul fizic „gândesc” diferit: la browser axa Y crește în jos, iar în fizică axa Y crește în sus. De aceea fiecare poziție trebuie „întoarsă” (Y-flip) la conversie, altfel obiectele ar cădea în sus. Regula de bază este simplă: **1 metru = 80 de pixeli**.

Doua sisteme de coordonate diferite



Conversie: 1 metru = 80 pixeli • $\text{canvas_y} = \text{sol} - \text{planck_y} \times 80$ (Y-flip)

Figura 2 - Diferența dintre coordonatele browserului și cele ale fizicii.

3.3. De ce simularea este stabilă (sub-pași)

În loc să avanseze fizica o singură dată pe cadru, motorul împarte fiecare cadru în **32 de sub-pași** mici. Cu pași mari, obiectele rapide ar putea „trece prin pereți” sau scena ar putea „exploda” numeric; cu pași mici, totul este mult mai precis. Există în plus o limitare de viteză la 200 m/s, ca plasă de siguranță.

3.4. Frecare și forță normală reale

Un detaliu important: valorile pentru forța normală (N) și frecare (Ff) afișate pe ecran **nu sunt estimări**, ci sunt **măsurate efectiv** din simulare. Motorul adună la fiecare contact impulsurile reale și le împarte la timp pentru a obține forța medie în Newtoni. Așa, un corp în repaus dă exact $N = m \cdot g$, cum trebuie.

3.5. Sfori, resorturi și scripeți

- **Sfoara** trage, dar nu împinge (limitează distanța maximă).
- **Resortul** folosește legea lui Hooke $F = k \cdot \Delta l$, cu o mică amortizare, ca un resort real.
- **Scripetele** trece sfoara peste un punct fix (ca la mașina Atwood).

Pentru realism, pendulele se opresc din cauza unei mici frecări la pivot (nu doar din cauza aerului), exact ca în viața reală.

3.6. Proiectul se autoverifică

Pentru a fi sigur că fizica e corectă, proiectul rulează automat „testul Atwood”: compară accelerația măsurată cu formula teoretică $a = (m_1 - m_2) \cdot g / (m_1 + m_2)$. Dacă diferența este foarte mică, testul trece. Asta arată că proiectul a fost construit cu grijă pentru **corectitudine**, nu doar ca să arate bine.

4. Desenare, grafice și export de date

Totul se desenează manual pe Canvas, la fiecare cadru: cerul, solul cu iarbă, grila cu metri, obiectele și **vectorii de forță colorați** (greutatea cu roșu, normala cu verde, frecarea cu portocaliu etc.), fiecare cu valoarea în Newtoni scrisă alături.

Simulatorul poate desena aproximativ **50 de grafice** diferite (viteză, accelerație, energie, forțe, plus categorii dedicate frecării și rezistenței aerului), care se actualizează în timp real. Datele pot fi apoi **exportate**:

- **Excel (.xlsx)** - toate seriile pe aceeași coloană de timp, gata de făcut grafice în Excel.
- **PNG** - toate graficele într-o singură imagine, gata de pus în Word sau PowerPoint.
- **JSON** - pentru salvarea și reîncărcarea experimentelor.



Captura 2 - Grafic generat de simulator: viteza totală în funcție de timp (linie dreaptă = mișcare uniform accelerată).

5. Interfața și ce poate face utilizatorul

- **Două moduri:** Simplu (doar masă și gravitație) și Avansat (toate controalele și graficele).
- **Catalog de obiecte:** ~30 de preseturi (cub, sferă, gheață, cauciuc, bilă de oțel, rampe), fiecare cu masă și frecare realiste.
- **Experimente gata făcute:** frecare, cădere liberă, plan înclinat, Atwood, resort, parașută.
- **13 probleme rezolvate pas cu pas:** pui datele (masă, unghi, μ) și primești rezolvarea cu formule și scena construită automat.
- **5 teme vizuale** și controale cu mouse, touch (pinch-zoom) și taste rapide (Space = play/pauză, R = reset).



Captura 3 - Panoul de control (modul Avansat): masă, frecare μ , forță aplicată, rezistența aerului și sub-pași.

6. Cum se construiește și se rulează

Codul scris în TypeScript este verificat și împachetat de Vite într-un singur fișier HTML, care apoi se deschide direct în browser, fără niciun server:

Cum se construiește proiectul

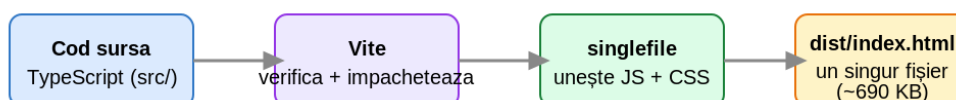


Figura 3 - Drumul de la codul sursă până la fișierul final.

`npm install` - instalează dependențele (o singură dată).

`npm run dev` - pornește modul de lucru, cu reîncărcare automată.

`npm run build` - construiește versiunea finală (dist/index.html).

7. Concluzie

Pe scurt, proiectul este un **referat de fizică despre frecare și rezistența aerului**, dublat de un **simulator 2D interactiv** scris în TypeScript, care folosește motorul Planck.js pentru coliziuni și adaugă deasupra fizica corectă de școală: forțe măsurate real, frecare, rezistența aerului, sfori, resorturi și scripeți. Totul este desenat pe Canvas, are circa 50 de grafice și export în Excel, și se împachetează într-un singur fișier care rulează în orice browser. Combină astfel **teoria scrisă** cu **experimentul interactiv**, într-un singur loc.